

Blockchat-Dao (Token)

SMART CONTRACT

Security Audit

Performed on Contracts:

Blockchat.sol

MD5 Hash: 1e7611eb02f09ed964bc13a1ece3bce1

Platform

METIS ANDROMEDA

hashlock.com.au

May 2024

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Behaviours	9
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Audit Findings	16
Centralisation	17
Conclusion	18
Our Methodology	19
Disclaimers	21
About Hashlock	22

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The BlockChatDao team partnered with Hashlock to conduct a security audit of their smart contract. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

BlockChat DAO is a communication and collaboration platform built for crypto enthusiasts. The platform combines messaging, social networking, forum creation, and BlockSpaces; where projects can update their community on key project milestones, host BlockParties, and a lot more features.

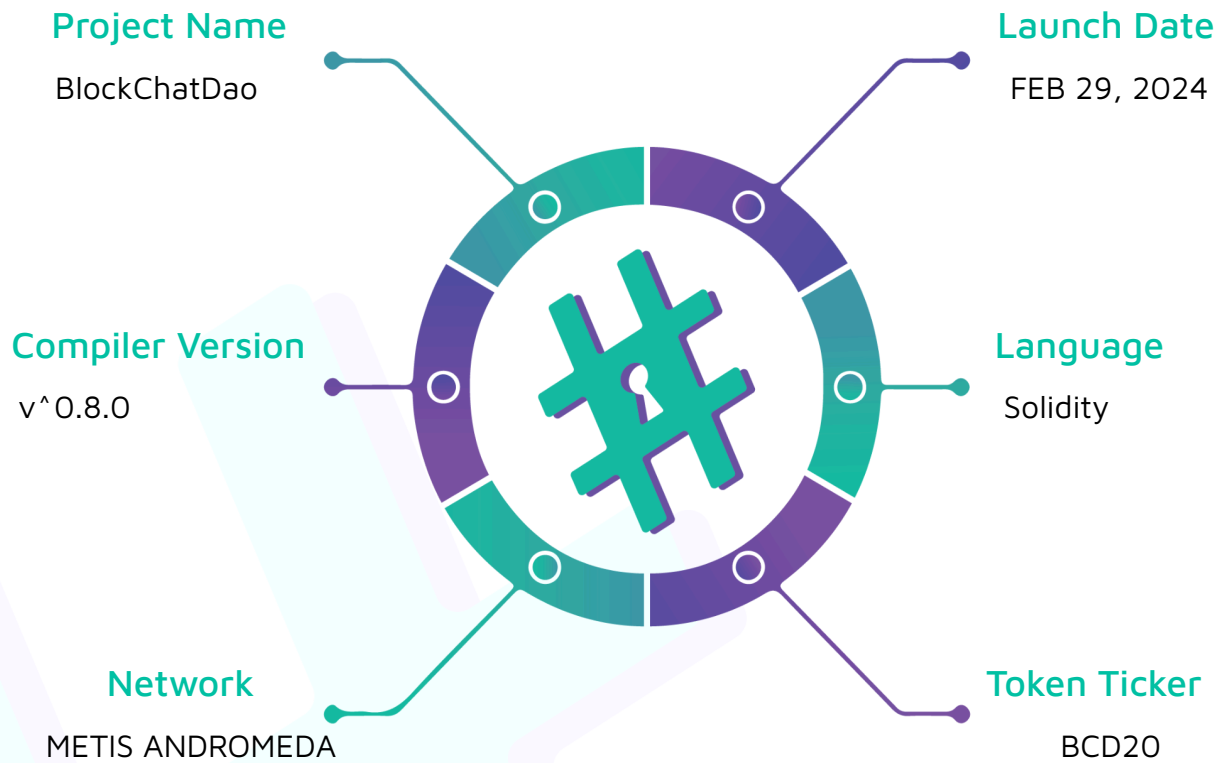
Project Name: BlockChatDao

Compiler Version: ^0.8.0

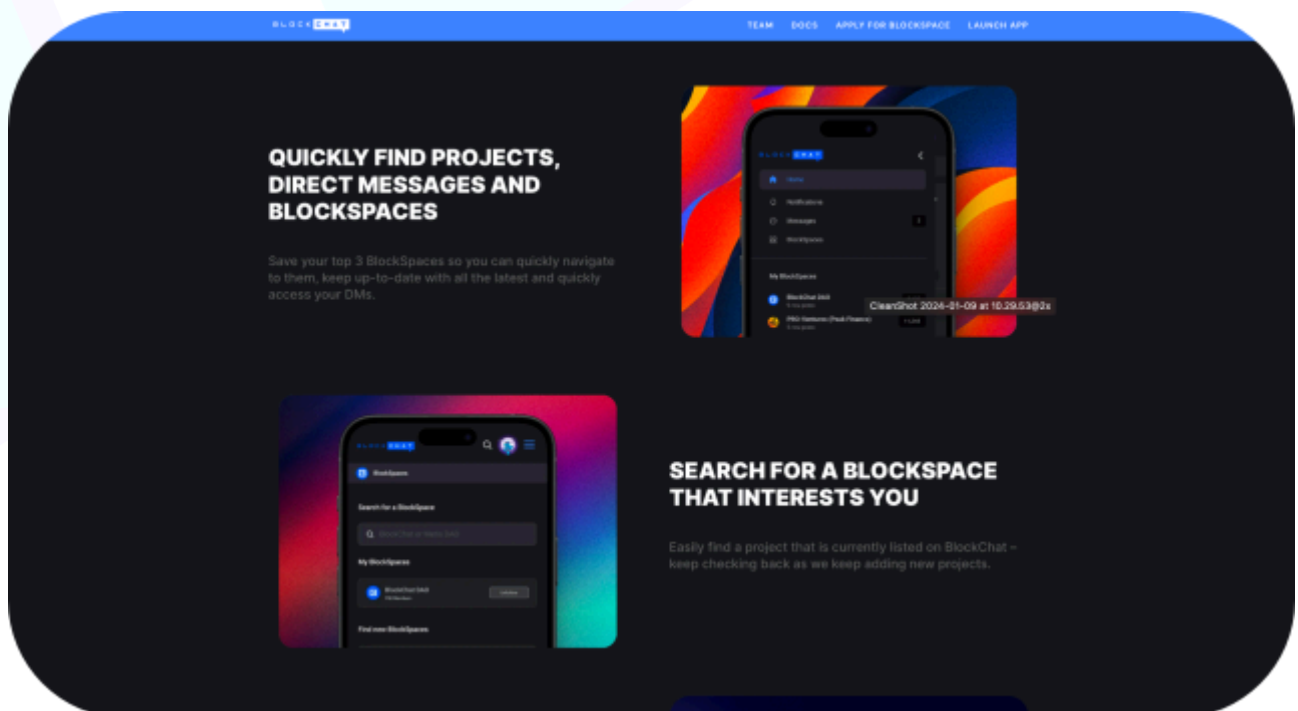
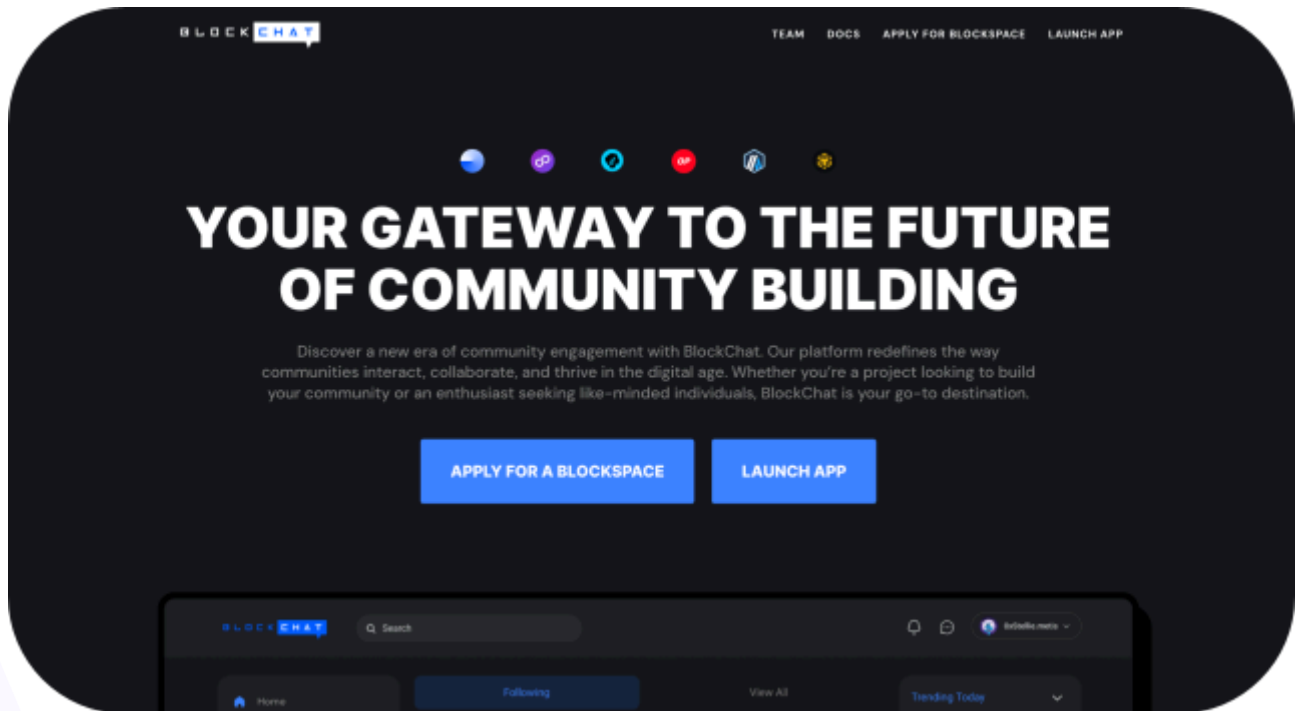
Website: blockchatdao.com

Logo:



Visualised Context:

Project Visuals:



#Hashlock.

Hashlock Pty Ltd

Audit scope

We at Hashlock audited the solidity code within the BlockChatDao project, the scope of works included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line by line analysis and were supported by software assisted testing.

Description	BlockChatDao Protocol Smart Contracts
Platform	Metis Andromeda / Solidity
Audit Date	May, 2024
Contract 1	Blockchat.sol
Contract 1 MD5 Hash	d215f44138a69b185b020a4044f194c0

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that need to be addressed prior to launch.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section.

All vulnerabilities we have identified have been resolved or acknowledged.

Hashlock found:

1 Medium severity vulnerabilities

3 Low severity vulnerabilities

3 Gas Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour
-------------------	------------------



Hashlock Pty Ltd

Context.sol • Purpose: Provides base functionalities for other contracts to extend, particularly regarding transaction context. • Key Features: ◦ <code>_msgSender()</code>: Returns the address of the caller of the function. ◦ <code>_msgData()</code>: Returns the data of the transaction that called the function.	Contract achieves this functionality.
Ownable.sol • Purpose: Adds ownership control to a contract, with specific functions only accessible by the owner. • Key Features: ◦ <code>onlyOwner</code> modifier: Restricts function access to the owner of the contract. ◦ <code>transferOwnership()</code>: Allows the current owner to transfer control of the contract to a new owner. ◦ <code>renounceOwnership()</code>: Enables the current owner to renounce their ownership, leaving the contract without an owner.	Contract achieves this functionality.
IERC20.sol • Purpose: Standard interface for ERC20 tokens, defining the necessary functions and events for	Contract achieves this functionality

token management. • Key Features: - Token balance and transfer functions (<code>balanceOf</code>, <code>transfer</code>). - Allowance and approval mechanisms (<code>approve</code>, <code>allowance</code>, <code>transferFrom</code>). - Events for transfer and approval tracking (<code>Transfer</code>, <code>Approval</code>).	
IERC20Metadata.sol • Purpose: Extension of the IERC20 interface that includes optional metadata functions. • Key Features: - Access to token metadata (<code>name</code>, <code>symbol</code>, <code>decimals</code>).	Contract achieves this functionality
ERC20.sol • Purpose: Implements the IERC20 interface, providing a complete token contract with customizable name, symbol, and optional supply mechanisms. • Key Features: - Standard ERC20 functions and internal mechanisms for token transfers (<code>_transfer</code>), approvals (<code>_approve</code>), and balance checks.	Contract achieves this functionality

○ Minting and burning capabilities (<code>_mint</code>, <code>_burn</code>). ○ Hooks for additional logic before and after transfers (<code>_beforeTokenTransfer</code>, <code>_afterTokenTransfer</code>).	
BlockChat.sol ● Purpose: Custom ERC20 token with additional taxation functionalities for transactions to certain addresses (like liquidity pools). ● Key Features: ○ Taxation on sells when tokens are transferred to specified automated market maker (AMM) pairs. ○ Distribution of collected taxes to specific wallets (liquidity, marketing, and development). ○ Adjustable taxation and wallet addresses through owner-restricted functions. ○ Control over which addresses are recognized as AMM pairs through <code>setAutomatedMarketMakerPair</code>.	Contract achieves this functionality

Code Quality

This Audit scope involves the smart contracts of the BlockChatDao project, as outlined in the Audit Scope section. All contracts, libraries and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however some refactoring is required.

The code is very well commented and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the BlockChatDao project's smart contract code in the form of a Meta explorer Link

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in understanding the overall architecture of the protocol.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies

Audit Findings

Medium

[M-01] - No way to transfer Ownership

Description

There isn't any way to transfer the ownership of the contract

Vulnerability Details

Currently, in the contract, multiple functions use "onlyOwner" modifier but the issue is that if the owner account is compromised then the malicious user can do various malicious actions and the owner can't do anything even if he knows that his account is compromised

```
function setLiquidityWallet(address _liquidityWallet) external onlyOwner {  
  
function setMarketingWallet(address _marketingWallet) external onlyOwner {  
  
function setDevelopmentWallet(address _developmentWallet) external onlyOwner {  
  
function setSellTax(uint256 _sellTax) external onlyOwner {  
  
function setLiquidityTax(uint256 _liquidityTax) external onlyOwner {  
  
function setMarketingTax(uint256 _marketingTax) external onlyOwner {  
  
function setDevelopmentTax(uint256 _devTax) external onlyOwner {  
  
function setAutomatedMarketMakerPair(address pair, bool enabled) public onlyOwner {
```

Impact

A malicious user can compromise the whole contract and possibly do a lot of unwanted things

Recommendation

It is recommended to add a "transferOwnership" functionality

Status

Resolved

Low**[L-01] Contracts** - Outdated version of Solidity**Description**

The project uses `pragma solidity ^0.8.0`. 0.8.0 is an outdated version of Solidity

Recommendation

Use `pragma solidity 0.8.25`.

Status

Acknowledged

[L-02] BlockChat - No check on address(0)**Description**

In the constructor of the contract, there isn't any kind of check on any of the address parameters so if by mistake any of these values gets set to address(0) then the contract has to be deployed again

Recommendation

Make sure to check for the address(0)

Status

Acknowledged

Gas

[G-01] Contracts - Use assembly to check for address(0)

Description

The project 0 address check to check if the address id 0 but the gas cost can be saved by using assembly

Using assembly can save 6 gas per instance

[G-02] Contracts - Use Custom Errors

Description

Instead of using error strings, to reduce deployment and runtime cost, you should use Custom Errors. This would save both deployment and runtime cost.

Source: <https://soliditylang.org/blog/2021/04/21/custom-errors/>

[G-03] Contracts - Use private rather than public for constants

Description

If needed, the values can be read from the verified contract source code, or if there are multiple values there can be a single getter function that [returns a tuple](#) of the values of all currently-public constants. Saves 3406-3606 gas in deployment gas due to the compiler not having to create non-payable getter functions for deployment call data, not having to store the bytes of the value outside of where it's used, and not adding another entry to the method ID table

Centralisation

The BlockChatDao project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the BlockChatDao project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and/or acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#Hashlock.

#Hashlock.

Hashlock Pty Ltd